



slington college
(इस्लिङ्टन कलेज)

Module Code & Module Title
Level 6 – Data and Web Development
Assessment Type
Semester 2025/26 Autumn

Student Name: Simanka Paudel

London Met ID: 23050271

College ID: NP01CP4A230098

Assignment Due Date: 09/03/2026

Assignment Submission Date: 09/03/2026

Submitted To: Ms Shresha Rajbhandari

Word Count:

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submissions and a mark of zero will be awarded.

Table of Contents

1	Introduction	1
2	Textual analysis	1
3	ERD from case study	3
4	Normalization	4
5	Integration and assumption	5
5.1	Resolution of Many-to-Many Relationships	5
5.2	Entities Combined During Integration	6
5.3	Additional Design Assumptions	6
5.4	Final Integrated Entities	7
6	Final ERD	8
7	Data dictionary	8
7.1	User table	8
7.2	Movie table	9
7.3	Theatre table	10
7.4	Hall table	10
7.5	Show table	11
7.6	Ticket table	11
7.7	User_Movie table	12
7.8	Theater_Movie_User table	12
7.9	HTMU table	13
7.10	SHTMU table	13
7.11	TSHTMU table	14
8	Script	15
8.1	User table	15
8.2	Movie table	16
8.3	Theater table	16
8.4	Hall table	17
8.5	Show table	17
8.6	Ticket table	17
8.7	User_Movie table	18
8.8	Theater_Movie_User table	19
8.9	HTMU table	20

8.10	SHTMU table	21
8.11	TSHTMU.....	23
9	Insert and select statement	24
1.1.1		24
10	Forms	35
10.1	Home page	35
10.2	Complex forms and queries	35
10.2.1	SQL Queries.....	35
10.2.2	Complex forms	37
10.2.3	Simple forms	38
11	User manual	39
12	Testing.....	42
12.1	User form	42
12.1.1	Insert	42
12.1.2	Update.....	43
12.1.3	Delete	43
12.2	Movie form	44
12.2.1	Insert	44
12.2.2	Update.....	45
12.2.3	Delete	46
12.3	Theater form	46
12.3.1	Insert	46
12.3.2	Update.....	47
12.3.3	Delete	48
12.4	Hall form	48
12.4.1	Insert	48
12.4.2	Update.....	49
12.4.3	Delete	50
12.5	Complex forms.....	50
12.5.1	Complex form 1	50
12.5.2	Complex form 2	51
12.5.3	Complex form 3	51

13	Further discussion	52
----	--------------------------	----

Table of figures

Figure 1: Initial ERD	4
Figure 2: User table script	16
Figure 3: Movie table script	16
Figure 4: Theater table	16
Figure 5: Hall table script.....	17
Figure 6: Show table	17
Figure 7: Ticket table script:	17
Figure 8: User_Movie table script.....	18
Figure 9: Theater_Movie_User table	19
Figure 10: Theater_Movie_User table	19
Figure 11: HTMU table script	20
Figure 12: HTMU table script	20
Figure 13: SHTMU table script	21
Figure 14: SHTMU table script	22
Figure 15: TSHTMU table script.....	23
Figure 16: TSHTMU table script.....	24
Figure 17: Inserting data into 'USERS' table	24
Figure 18: Inserting data into 'USERS' table	25
Figure 19: Output of 'USERS' table	25
Figure 20: Insertion of data in 'Movie' table	25
Figure 21: Update on 'Movie' table	26
Figure 22: Output of Movie table	26
Figure 23: Insertion of data in 'Theatre' table	27
Figure 24: Output of Theatre table	27
Figure 25: Insertion of data in 'Hall' table	28
Figure 26: Output of 'Hall' table	28
Figure 27: Inserting data into 'Show' table.....	28
Figure 28: Inserting of data in Show table	29
Figure 29: Output of 'Show' table	29
Figure 30: Inserting of data in Ticket table	30
Figure 31: Output of 'Ticket' table.....	30
Figure 32: Inserting data into 'User_Movie' table	31
Figure 33: Output of User_Movie table.....	31
Figure 34: Insertion of data in 'Theater_Movie_User' table	31
Figure 35: Insertion of data	32
Figure 36: Outout of 'Theater_Movie_User'	32
Figure 37: Inserting data into 'HTMU' table	32
Figure 38: Insert data into 'HTMU' table	33
Figure 39: Output of 'HTMU' table	33
Figure 40: Inserting data into 'SHTMU' table	34
Figure 41: Output of 'SHTMU' table.....	34
Figure 42: Inserting data into 'TSHTMU' table.....	34

Figure 43: Output of 'TSHTMU' table	34
Figure 44: Home page.....	35
Figure 45: Complex query 1	36
Figure 46: Complex Query 2	36
Figure 47: Complex Query 3	37
Figure 48: Complex form 1	37
Figure 49: Complex form 1	37
Figure 50: Complex form 1	37
Figure 51: Complex form 2	37
Figure 52: Complex form 2	38
Figure 53: Complex form 2	38
Figure 54: Complex form 3	38
Figure 55: Complex form 3	38
Figure 56: Complex form 3	38
Figure 57: User details form	38
Figure 58: Show details form.....	39
Figure 59: Movie details form	39
Figure 60: Ticket details form	39
Figure 61: User list	40
Figure 62: User form data insertion	40
Figure 63: Movie form	40
Figure 64: Inserting into movie form	41
Figure 65: Theater form list	41
Figure 66: Insertion of new theater data.....	41
Figure 67: User form before insertion.....	42
Figure 68: Inserting data in user form.....	42
Figure 69: User form after data insertion.....	43
Figure 70: User form before update	43
Figure 71: User form during update.....	43
Figure 72: User form after update	43
Figure 73: User form before deletion.....	44
Figure 74: User form after deletion.....	44
Figure 75: Movie form before insertion	44
Figure 76: Movie form during insertion	45
Figure 77: Movie form after insertion	45
Figure 78: Movie form before update	45
Figure 79: Movie form during update.....	45
Figure 80: Movie form after update	46
Figure 81: Movie form before deletion.....	46
Figure 82: Theater form before insertion	46
Figure 83: Theater form during insertion	47
Figure 84: Theater form after insertion	47
Figure 85: Theater form before update.....	47

Figure 86: Theater form during update	47
Figure 87: Theater form after update.....	48
Figure 88: Theater form before deletion	48
Figure 89: Theater form after deletion	48
Figure 90: Hall form before insertion	49
Figure 91: Hall form during insertion	49
Figure 92: Hall form after new insertion.....	49
Figure 93: Hall form before update.....	49
Figure 94: Hall form during update	49
Figure 95: Hall form after insertion	50
Figure 96: Hall form before deletion	50
Figure 97: Complex form 1	50
Figure 98: Complex form 1	50
Figure 99: Complex form 1	51
Figure 100: Complex form 2	51
Figure 101: Complex form 2	51
Figure 102: Complex form 2	51
Figure 103: Complex form 2	51
Figure 104: Complex form 3	51
Figure 105: Complex form 3	51
Figure 106: Complex form 3	52

Table of tables

Table 1: User table data dictionary.....	8
Table 2: Movie table	9
Table 3: Theatre table	10
Table 4: Hall table	11
Table 5: Show table.....	11
Table 6: Ticket table	12
Table 7: User_Movie table	12
Table 8: Theater_Movie_User table	13
Table 9: HTMU table	13
Table 10: SHTMU table	14
Table 11: TSHTMU table.....	15

1 Introduction

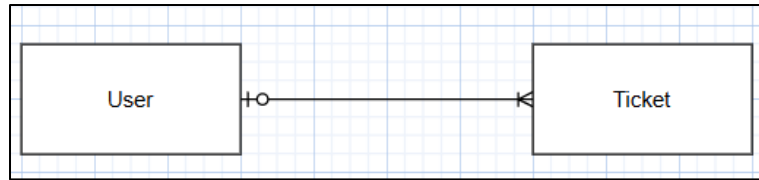
This coursework focuses on the design and development of a database-based web application for a cinema management system using the case study of KumariCinemas. KumariCinemas is planning to expand its cinema chain across different cities in Nepal, but the organization currently faces several operational challenges. These include issues related to managing theatre locations, hall capacities, ticket bookings, and maintaining an efficient communication system. In addition, existing cinema halls often struggle with poor customer experience and inefficient ticket handling. To overcome these problems, KumariCinemas plans to implement a centralized digital system that will help manage ticket sales, automate several operational processes, and provide a smoother movie booking experience for customers.

The main objective of this project is to analyze the requirements of the given case study and design a structured database system that supports the cinema's operations. The project involves identifying the relevant data, organizing it through the process of normalization up to the Third Normal Form (3NF), and designing Entity Relationship Diagrams (ERDs) that represent the relationships between different entities in the system. The database is implemented using Oracle SQL Developer, where tables are created and populated with sample data. In addition, a web-based application is developed using ASP.NET with C# to allow users to interact with the system through different forms such as user information, movie details, theatre and hall details, showtimes, and ticket management. The system also includes more advanced features such as displaying user ticket history, theatre movie schedules, and identifying theatres with the highest seat occupancy for a particular movie.

2 Textual analysis

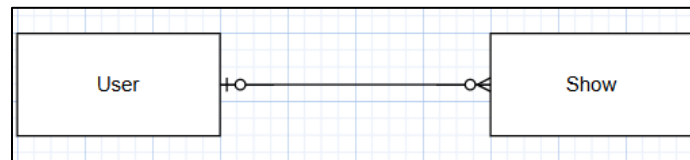
- **User and Ticket:**

A single user can purchase one or several tickets, but each ticket is associated with only one user.



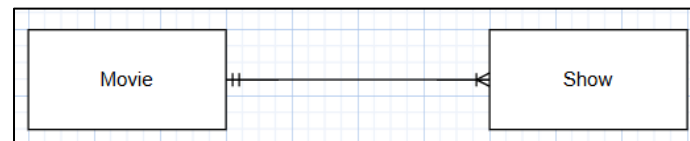
- **User and Show:**

A user may book one or multiple shows, and a show can also be booked by many different users.



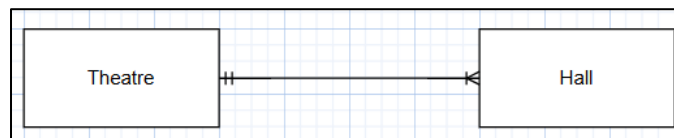
- **Movie and Show:**

A movie may be scheduled for one or several shows, while each show is linked to only one movie.



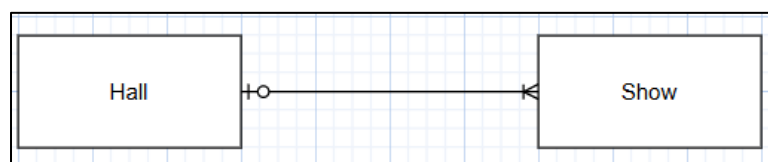
- **Theatre and Hall:**

A theatre can contain multiple halls, but every hall is part of only one theatre.



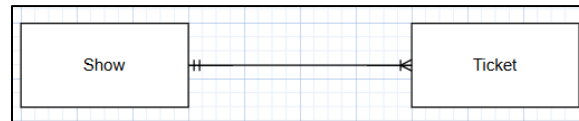
- **Hall and Show:**

A hall can host many shows, but each show takes place in only one hall.



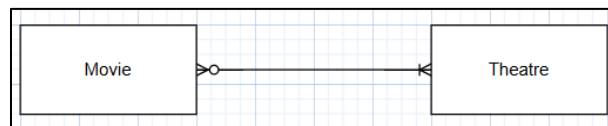
- **Show and Ticket:**

A show can generate multiple tickets, while every ticket is issued for a single show.



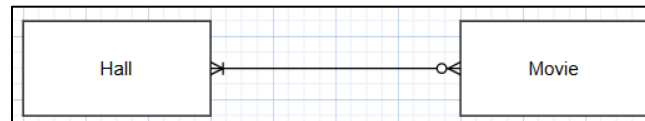
- **Movie and Theatre:**

A movie may be screened in several theatres, and each theatre can screen many different movies.



- **Hall and Movie:**

A hall can be used to screen multiple movies, and a movie may be shown in more than one hall.



3 ERD from case study

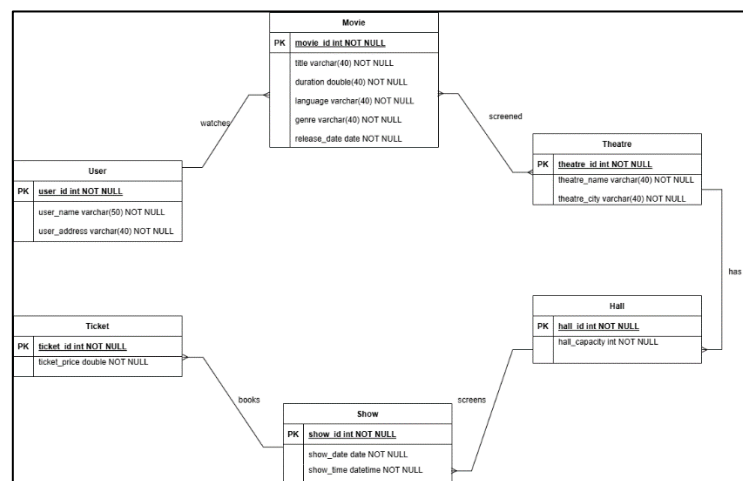


Figure 1: Initial ERD

4 Normalization

UNF (Unnormalized Form)

RD{RG}

UserId, Username, Address, {MovieId, Title, Duration, Language, Genre, ReleaseDate, {TheatreId, TheatreName, TheatreCity, {HallId, HallName, HallCapacity, {ShowId, ShowDate, ShowTimes, {TicketId, TicketPrice} }}}} }

1NF

User(UserId, Username, Address)

Movie(MovieId, Title, Duration, Language, Genre, ReleaseDate, UserId*)

Theatre(TheatreId, TheatreName, TheatreCity, MovieId*, UserId*)

Hall(HallId, HallName, HallCapacity, TheatreId*, MovieId*, UserId*)

Show(ShowId, ShowDate, ShowTimes, HallId*, TheatreId*, MovieId*, UserId*)

Ticket(TicketId, TicketPrice, HallId*, TheatreId*, MovieId*, UserId*)

2NF

Removing partial dependencies:

User(UserId, Username, Address)

Movie(MovieId, Title, Duration, Language, Genre, ReleaseDate)

User_Movie(UserId*, MovieId*)

Theatre(TheatreId, TheatreName, TheatreCity)

Theatre_Movie_User(TheatreId*, MovieId*, UserId*)

Hall(HallId, HallName, HallCapacity)

Hall_Theatre_Movie_User(HallId*, TheatreId*, MovieId*, UserId*)

Show(ShowId, ShowDate, ShowTimes)

Show_Hall_Theatre_Movie_User(ShowId*, HallId*, TheatreId*, MovieId*, UserId*)

Ticket(TicketId, TicketPrice)

Ticket_Show_Hall_Theatre_Movie_User(ShowId*, HallId*, TheatreId*, MovieId*, UserId*)

3NF

$K \rightarrow NK \rightarrow NK$ (Removing transitive dependencies)

User(UserId, Username, Address)

Movie(MovieId, Title, Duration, Language, Genre, ReleaseDate)

User_Movie(UserId*, MovieId*)

Theatre(TheatreId, TheatreName, TheatreCity)

Theatre_Movie_User(TheatreId*, MovieId*, UserId*)

Hall(HallId, HallName, HallCapacity)

Hall_Theatre_Movie_User(HallId*, TheatreId*, MovieId*, UserId*)

Show(ShowId, ShowDate, ShowTime)

Show_Hall_Theatre_Movie_User(ShowId*, HallId*, TheatreId*, MovieId*, UserId*)

Ticket(TicketId, TicketPrice)

Ticket_Show_Hall_Theatre_Movie_User(ShowId*, HallId*, TheatreId*, MovieId*, UserId*)

5 Integration and assumption

5.1 Resolution of Many-to-Many Relationships

During the normalization process, the many-to-many relationships identified in the original ERD were eliminated by introducing junction (bridge) tables.

Relationship	Resolved Using
Movie – Theatre (many-to-many)	Theatre_Movie_User
Movie – Hall (many-to-many)	Hall_Theatre_Movie_User
User – Movie (many-to-many)	User_Movie
Show – Hall (many-to-many)	Show_Hall_Theatre_Movie_User

These junction tables help convert many-to-many relationships into multiple one-to-many relationships, ensuring the database structure remains properly normalized.

5.2 Entities Combined During Integration

While integrating the normalized tables into the final schema, several relationships from the initial ERD were represented through the following tables:

Original Relationship	Represented By
User watches Movie	User_Movie
Movie screened at Theatre	Theatre_Movie_User
Theatre contains Hall	HTMU
Hall hosts Show	SHTMU
Show generates Ticket	TSHTMU

5.3 Additional Design Assumptions

Assumption 1: TicketStatus introduced for cancellation handling

The case study indicates that if a ticket is not purchased at least one hour before the show begins, it should be cancelled automatically. To support this rule, a TicketStatus attribute was included in the Ticket entity.

Ticket(TicketId, TicketDate, TicketStatus)

Possible values for TicketStatus include:

- Booked
- Paid
- Cancelled

Assumption 3: HallName included for easier identification

A HallName attribute was introduced in the Hall entity to make each hall easier to distinguish.

Relying only on HallCapacity is not sufficient to uniquely identify halls within the same theatre.

Assumption 4: TicketId included in the final junction table

In the 3NF design, the table Ticket_Show_Hall_Theatre_Movie_User did not originally include TicketId as a foreign key. This was corrected in the final ERD to properly link ticket records.

Ticket_Show_Hall_Theatre_Movie_User

(TicketId*, ShowId*, HallId*, TheatreId*, MovieId*, UserId*)

5.4 Final Integrated Entities

The following entities form the final integrated database schema:

User(UserId PK, Username, Address)

Movie(MovieId PK, Title, Duration, Language, Genre, ReleaseDate)

Theatre(TheatreId PK, TheatreName, TheatreCity)

Hall(HallId PK, HallName, HallCapacity)

Show>ShowId PK, ShowDate, ShowTime)

Ticket(TicketId PK, TicketDate, TicketStatus)

User_Movie(UserId FK, MovieId FK)

Theatre_Movie_User(TheatreId FK, MovieId FK, UserId FK)

Hall_Theatre_Movie_User(HallId FK, TheatreId FK, MovieId FK, UserId FK)

Show_Hall_Theatre_Movie_User(ShowId FK, HallId FK, TheatreId FK, MovieId FK, UserId FK)

Ticket_Show_Hall_Theatre_Movie_User(TicketId FK, ShowId FK, HallId FK, TheatreId FK, MovieId FK, UserId FK)

6 Final ERD

7 Data dictionary

7.1 User table

Column name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example data
USER_ID	NUMBER	10	Primary Key			Unique identifier assigned to each user	1
USER_NAME	VARCHAR2	50	Not Null			Stores full name of the registered user	A
ADDRESS	VARCHAR2	40	Not Null			Stores the residential address of the user	60

Table 1: User table data dictionary

7.2 Movie table

Column name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example data
Movie_Id	NUMBER	10	Primary Key			Unique identifier for each movie	1
Title	VARCHAR2	50	Not Null			Name or title of the movie	Mean girls
Duration	NUMBER	5	Not Null			Length of the movie measured in hours	1.5
Language	VARCHAR2	40	Not Null			Language in which the movie is available	English
Genre	VARCHAR2	40	Not Null			Category or type of the movie	Rom-Com
Release_date	DATE	-	Not Null			Official date when the movie was released	05-JAN-26

Table 2: Movie table

7.3 Theatre table

Column name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example data
Theater_Id	NUMBER	10	Primary Key			Unique identifies each theatre	1
Theater_Name	VARCHAR2	40	Not Null			Stores the name of the theatre	Kumari Complex
Theater_City	VARCHAR2	30	Not Null			Stores the city where theatre is located	60

Table 3: Theatre table

7.4 Hall table

Column name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example data
Hall_Id	NUMBER	10	Primary Key			Uniquely identifies each hall	1
Hall_Name	VARCHAR2	40	Not Null			Stores the name/label of the hall	A

Hall_Capacity	NUMBER	5	Not Null			Stores the seating capacity of the hall	60
---------------	--------	---	----------	--	--	---	----

Table 4: Hall table

7.5 Show table

Column name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example data
Show_Id	NUMBER	10	Primary Key			Uniquely identifies each show	1
Show_Date	DATE	-	Not Null			Stores the date of the show	15-Jan-26
Show_Time	DATE	-	Not Null			Stores the time of the show	07-Jun-04

Table 5: Show table

7.6 Ticket table

Column name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example data
Ticket_Id	NUMBER	10	Primary Key			Uniquely identifies each ticket	1

Ticket_Date	DATE	-	Not Null			Stores the date of the ticket was purchased	05-Jan-26
-------------	------	---	----------	--	--	---	-----------

Table 6: Ticket table

7.7 User_Movie table

Column name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example data
User_Id	NUMBER	10	Primary Key, Foreign Key	Users	User_Id	Reference s the user who watched movie	1
Movie_Id	NUMBER	10	Primary Key, Foreign Key	Movie	Movie_Id	Reference s movie watched by user	1

Table 7: User_Movie table

7.8 Theater_Movie_User table

Column name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example data
Theater_Id	NUMBER	10	Primary Key, Foreign Key	Theatre	Theater_Id	Reference s the theatre	1
Movie_Id	NUMBER	10	Primary Key,	Movie	Movie_Id	Reference s the movie	1

			Foreign Key				
User_Id	NUMBER	10	Primary Key, Foreign Key	Users	User_Id	References the user	1

Table 8: Theater_Movie_User table

7.9 HTMU table

Column name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example data
Hall_Id	NUMBER	10	Primary Key, Foreign Key	Hall	Hall_Id	References the hall	1
Theater_Id	NUMBER	10	Primary Key, Foreign Key	Theatre	Theater_Id	References the theatre	1
Movie_Id	NUMBER	10	Primary Key, Foreign Key	Movie	Movie_Id	References the movie	1
User_Id	NUMBER	10	Primary Key, Foreign Key	Users	User_Id	References the user	2

Table 9: HTMU table

7.10 SHTMU table

Column name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example data
-------------	-----------	------	------------	-----------------	------------------	-------------	--------------

Show_Id	Number	10	Primary Key, Foreign Key	Show	Show_Id	Reference s the show	1
Hall_Id	NUMBER	10	Primary Key, Foreign Key	Hall	Hall_Id	Reference s the hall	2
Theater_Id	NUMBER	10	Primary Key, Foreign Key	Theatre	Theater_Id	Reference s the theatre	5
Movie_Id	NUMBER	10	Primary Key, Foreign Key	Movie	Movie_Id	Reference s the movie	5
User_Id	NUMBER	10	Primary Key, Foreign Key	Users	User_Id	Reference s the user	4

Table 10: SHTMU table

7.11 TSHTMU table

Column name	Data Type	Size	Constraint	Reference Table	Reference Column	Description	Example data
Ticket_Id	NUMBER	10	Primary Key, Foreign Key	Ticket	Ticket_Id	Reference s the ticket	1

Show_Id	NUMBER	10	Primary Key, Foreign Key	Show	Show_Id	References the show	1
Hall_Id	NUMBER	10	Primary Key, Foreign Key	Hall	Hall_Id	References the hall	2
Theater_Id	NUMBER	10	Primary Key, Foreign Key	Theatre	Theater_Id	References the theatre	5
Movie_Id	NUMBER	10	Primary Key, Foreign Key	Movie	Movie_Id	References the movie	5
User_Id	NUMBER	10	Primary Key, Foreign Key	Users	User_Id	References the user	4

Table 11: TSHTMU table

8 Script

8.1 User table

```
CREATE TABLE "User"
(
    user_id    INTEGER NOT NULL ,
    user_name  VARCHAR2 (50) ,
    address    VARCHAR2 (40)
)
;

ALTER TABLE "User"
ADD CONSTRAINT User_PK PRIMARY KEY ( user_id ) ;
```

Figure 2: User table script

8.2 Movie table

```
DDL Preview
CREATE TABLE Movie
(
    movie_id    INTEGER NOT NULL ,
    title       VARCHAR2 (50) ,
    language    VARCHAR2 (40) ,
    duration     NUMBER ,
    genre        VARCHAR2 (40) ,
    release_date DATE
)
;

ALTER TABLE Movie
    ADD CONSTRAINT Movie_PK PRIMARY KEY ( movie_id ) ;
```

Figure 3: Movie table script

8.3 Theater table

```
DDL Preview
CREATE TABLE Theatre
(
    theater_id  INTEGER NOT NULL ,
    theater_name VARCHAR2 (40) ,
    theater_city VARCHAR2 (30)
)
;

ALTER TABLE Theatre
    ADD CONSTRAINT Theatre_PK PRIMARY KEY ( theater_id ) ;
```

Figure 4: Theater table

8.4 Hall table

```
CREATE TABLE Hall
(
    hall_id      INTEGER NOT NULL ,
    hall_capacity INTEGER ,
    hall_name    VARCHAR2 (40)
)
;

ALTER TABLE Hall
    ADD CONSTRAINT Hall_PK PRIMARY KEY ( hall_id ) ;
```

Figure 5: Hall table script

8.5 Show table

```
DDL Preview
1 CREATE TABLE Show
2
3     (
4         show_id  INTEGER NOT NULL ,
5         show_date DATE ,
6         show_time DATE
7     )
8
9     ;
10
11 ALTER TABLE Show
12     ADD CONSTRAINT Show_PK PRIMARY KEY ( show_id ) ;
```

Figure 6: Show table

8.6 Ticket table

```
CREATE TABLE Ticket
(
    ticket_id  INTEGER NOT NULL ,
    ticket_date DATE
)
;

ALTER TABLE Ticket
    ADD CONSTRAINT Booking_PK PRIMARY KEY ( ticket_id ) ;
```

Figure 7: Ticket table script:

8.7 User_Movie table

```
CREATE TABLE User_Movie
(
    user_id INTEGER NOT NULL ,
    movie_id INTEGER NOT NULL
)
;

ALTER TABLE User_Movie
    ADD CONSTRAINT User_Movie_PK PRIMARY KEY ( user_id, movie_id ) ;

ALTER TABLE User_Movie
    ADD CONSTRAINT User_Movie_Movie_FK FOREIGN KEY
    (
        movie_id
    )
    REFERENCES Movie
    (
        movie_id
    )
;

ALTER TABLE User_Movie
    ADD CONSTRAINT User_Movie_User_FK FOREIGN KEY
    (
        user_id
    )
    REFERENCES "User"
    (
        user_id
    )
;
```

Figure 8: User_Movie table script

8.8 Theater_Movie_User table

```
DDL Preview
CREATE TABLE Theater_Movie_User
(
    theater_id INTEGER NOT NULL ,
    movie_id   INTEGER NOT NULL ,
    user_id    INTEGER NOT NULL
)

ALTER TABLE Theater_Movie_User
ADD CONSTRAINT Theater_Movie_User_PK PRIMARY KEY ( theater_id ) ;

ALTER TABLE Theater_Movie_User
ADD CONSTRAINT Theater_Movie_User_Movie_FK FOREIGN KEY
(
    movie_id
)
REFERENCES Movie
(
    movie_id
)
;

ALTER TABLE Theater_Movie_User
ADD CONSTRAINT Theater_Movie_User_Theatre_FK FOREIGN KEY
(
    theater_id
)
REFERENCES Theatre
(
    theater_id
)
;
```

Figure 9: Theater_Movie_User table

```
ALTER TABLE Theater_Movie_User
ADD CONSTRAINT Theater_Movie_User_Theatre_FK FOREIGN KEY
(
    theater_id
)
REFERENCES Theatre
(
    theater_id
)
;

ALTER TABLE Theater_Movie_User
ADD CONSTRAINT Theater_Movie_User_User_FK FOREIGN KEY
(
    user_id
)
REFERENCES "User"
(
    user_id
)
;
```

Figure 10: Theater_Movie_User table

8.9 HTMU table

```
DDL Preview
CREATE TABLE HTMU
(
    hall_id    INTEGER NOT NULL ,
    theater_id INTEGER NOT NULL ,
    movie_id   INTEGER NOT NULL ,
    user_id    INTEGER NOT NULL
)
;

ALTER TABLE HTMU
ADD CONSTRAINT Hall_Theater_Movie_User_PK PRIMARY KEY ( hall_id, theater_id, movie_id, user_id ) ;

ALTER TABLE HTMU
ADD CONSTRAINT HTMU_Hall_FK FOREIGN KEY
(
    hall_id
)
REFERENCES Hall
(
    hall_id
)
;
```

Figure 11: HTMU table script

```
ALTER TABLE HTMU
ADD CONSTRAINT HTMU_Theatre_FK FOREIGN KEY
(
    theater_id
)
REFERENCES Theatre
(
    theater_id
)
;

ALTER TABLE HTMU
ADD CONSTRAINT HTMU_User_FK FOREIGN KEY
(
    user_id
)
REFERENCES "User"
(
    user_id
)
;
```

Figure 12: HTMU table script

8.10 SHTMU table

```
CREATE TABLE SHTMU
(
    show_id    INTEGER NOT NULL ,
    hall_id    INTEGER NOT NULL ,
    theatre_id INTEGER NOT NULL ,
    movie_id   INTEGER NOT NULL ,
    user_id    INTEGER NOT NULL
)
;

ALTER TABLE SHTMU
    ADD CONSTRAINT SHTMUPK PRIMARY KEY ( show_id, hall_id, theatre_id, movie_id, user_id ) ;

ALTER TABLE SHTMU
    ADD CONSTRAINT SHTMU_Hall_FK FOREIGN KEY
    (
        hall_id
    )
    REFERENCES Hall
    (
        hall_id
    )
;

ALTER TABLE SHTMU
    ADD CONSTRAINT SHTMU_Movie_FK FOREIGN KEY
    (
        movie_id
    )
    REFERENCES Movie
    (
        movie_id
    )
;
```

Figure 13: SHTMU table script

```
ALTER TABLE SHTMU
  ADD CONSTRAINT SHTMU_Show_FK FOREIGN KEY
  (
    show_id
  )
  REFERENCES Show
  (
    show_id
  )
;

ALTER TABLE SHTMU
  ADD CONSTRAINT SHTMU_Theatre_FK FOREIGN KEY
  (
    theatre_id
  )
  REFERENCES Theatre
  (
    theater_id
  )
;

ALTER TABLE SHTMU
  ADD CONSTRAINT SHTMU_User_FK FOREIGN KEY
  (
    user_id
  )
  REFERENCES "User"
  (
    user_id
  )
;
```

Figure 14: SHTMU table script

8.11 TSHTMU

```
DDL Preview
CREATE TABLE TSHTMU
(
    ticket_id INTEGER NOT NULL ,
    show_id   INTEGER NOT NULL ,
    hall_id   INTEGER NOT NULL ,
    movie_id  INTEGER NOT NULL ,
    theater_id INTEGER NOT NULL ,
    user_id   INTEGER NOT NULL
)
;

ALTER TABLE TSHTMU
    ADD CONSTRAINT TSHTMU_PK PRIMARY KEY ( show_id, theater_id, ticket_id, movie_id, user_id, hall_id ) ;

ALTER TABLE TSHTMU
    ADD CONSTRAINT TSHTMU_Hall_FK FOREIGN KEY
    (
        hall_id
    )
    REFERENCES Hall
    (
        hall_id
    )
;

ALTER TABLE TSHTMU
    ADD CONSTRAINT TSHTMU_Movie_FK FOREIGN KEY
    (
        movie_id
    )
    REFERENCES Movie
    (
        movie_id
    )
;
```

Figure 15: TSHTMU table script

```
ALTER TABLE TSHTMU
  ADD CONSTRAINT TSHTMU_Ticket_FK FOREIGN KEY
  (
    ticket_id
  )
  REFERENCES Ticket
  (
    ticket_id
  )
;

ALTER TABLE TSHTMU
  ADD CONSTRAINT TSHTMU_User_FK FOREIGN KEY
  (
    user_id
  )
  REFERENCES "User"
  (
    user_id
  )
;
```

Figure 16: TSHTMU table script

9 Insert and select statement

1.1.1

```
SQL> INSERT INTO USERS(user_id,user_name,address)
2  VALUES(1,'Simanka','Thaiba');

1 row created.

SQL> INSERT INTO USERS(user_id,user_name,address)
2  VALUES(2,'Riwaj','Balaju');

1 row created.

SQL> INSERT INTO USERS(user_id,user_name,address)
2  VALUES(3,'Aasra','NewRoad');

1 row created.
```

Figure 17: Inserting data into 'USERS' table

```
SQL> INSERT INTO USERS(user_id,user_name,address)
2 VALUES(4,'Simran','Thamel');

1 row created.

SQL> INSERT INTO USERS(user_id,user_name,address)
2 VALUES(5,'Sam','Asan');

1 row created.
```

Figure 18: Inserting data into 'USERS' table

- Output:

```
SQL> SELECT * FROM USERS;
```

USER_ID	USER_NAME	ADDRESS
1	Simanka	Thaiba
2	Riwaj	Balaju
3	Aasra	NewRoad
4	Simran	Thamel
5	Sam	Asan

Figure 19: Output of 'USERS' table

- Movie Table

```
SQL> INSERT INTO Movie(movie_id,title,duration,genre,release_date)
2 VALUES(1, 'Mean girls', 1.5 , 'Rom-Com',TO_DATE('2002-01-05', 'YYYY-MM-DD'))
3 ;

1 row created.

SQL> INSERT INTO Movie(movie_id,title,duration,genre,release_date)
2 VALUES(2, 'It Ends with us', 2.5 , 'Romantic-Drama',TO_DATE('2026-01-15', 'YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Movie(movie_id,title,duration,genre,release_date)
2 VALUES(3, 'NoteBook', 2 , 'Rom-Com',TO_DATE('2026-01-08', 'YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Movie(movie_id,title,duration,genre,release_date)
2 VALUES(4, 'Batman', 2.5 , 'Action',TO_DATE('2026-01-04', 'YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Movie(movie_id,title,duration,genre,release_date)
2 VALUES(5, 'Harry Potter', 2 , 'Fantasy',TO_DATE('2026-01-10', 'YYYY-MM-DD'));

1 row created.
```

Figure 20: Insertion of data in 'Movie' table

```

SQL> UPDATE Movie
  2 SET language = 'English'
  3 WHERE movie_id = 1;

1 row updated.

SQL> UPDATE Movie
  2 SET language = 'English'
  3 WHERE movie_id = 2;

1 row updated.

SQL> UPDATE Movie
  2 SET language = 'English'
  3 WHERE movie_id = 3;

1 row updated.

SQL> UPDATE Movie
  2 SET language = 'English'
  3 WHERE movie_id = 4;

1 row updated.

SQL> UPDATE Movie
  2 SET language = 'English'
  3 WHERE movie_id = 5;

1 row updated.

```

Figure 21: Update on 'Movie' table

- Output

```
SQL> select * from Movie;
```

MOVIE_ID	TITLE	LANGUAGE	DURATION	GENRE	RELEASE_D
1	Mean girls	English	1.5	Rom-Com	05-JAN-02
2	It Ends with us	English	2.5	Romantic-Drama	15-JAN-26
3	NoteBook	English	2	Rom-Com	08-JAN-26
4	Batman	English	2.5	Action	04-JAN-26
5	Harry Potter	English	2	Fantasy	10-JAN-26

Figure 22: Output of Movie table

- Theater Table

```

SQL> INSERT INTO Theatre(theater_id,theater_name,theater_city)
2 VALUES(1,'Kumari Complex','KamalPokari');

1 row created.

SQL> INSERT INTO Theatre(theater_id,theater_name,theater_city)
2 VALUES(2,'QFX','Pulchowk');

1 row created.

SQL> INSERT INTO Theatre(theater_id,theater_name,theater_city)
2 VALUES(3,'One Cinemas','Kalimati');

1 row created.

SQL> INSERT INTO Theatre(theater_id,theater_name,theater_city)
2 VALUES(4,'One Cinemas','Baneshwor');

1 row created.

SQL> INSERT INTO Theatre(theater_id,theater_name,theater_city)
2 VALUES(5,'BishwoJyoti Cineplex','Jamal');

1 row created.

SQL> COMMIT;

Commit complete.

```

Figure 23: Insertion of data in 'Theatre' table

- Output

```

SQL> select * from Theatre;

```

THEATER_ID	THEATER_NAME	THEATER_CITY
1	Kumari Complex	KamalPokari
2	QFX	Pulchowk
3	One Cinemas	Kalimati
4	One Cinemas	Baneshwor
5	BishwoJyoti Cineplex	Jamal

Figure 24: Output of Theatre table

- Hall Table

```

SQL> INSERT into Hall(hall_id,hall_name,hall_capacity)
  2  VALUES(1, 'A', 60);

1 row created.

SQL> INSERT into Hall(hall_id,hall_name,hall_capacity)
  2  VALUES(2, 'B', 80);

1 row created.

SQL> INSERT into Hall(hall_id,hall_name,hall_capacity)
  2  VALUES(3, 'C', 100);

1 row created.

SQL> INSERT into Hall(hall_id,hall_name,hall_capacity)
  2  VALUES(4, 'D', 110);

1 row created.

SQL> INSERT into Hall(hall_id,hall_name,hall_capacity)
  2  VALUES(5, 'E', 120);

1 row created.

SQL> COMMIT;

Commit complete.

```

Figure 25: Insertion of data in "Hall" table

- Output

```

SQL> select * from hall;

  HALL_ID HALL_CAPACITY HALL_NAME
-----
        1          60 A
        2          80 B
        3         100 C
        4         110 D
        5         120 E

```

Figure 26: Output of 'Hall' table

- Show table

```

SQL> INSERT into Show(show_id,show_date,show_time)
  2  VALUES(1,TO_DATE('2004-06-07', 'YYYY-MM-DD'),TO_TIMESTAMP('2004-06-07 18:30:00', 'YYYY-MM-DD HH24:MI:SS'));

1 row created.

```

Figure 27: Inserting data into 'Show' table

```

SQL> INSERT into Show(show_id,show_date,show_time)
      2 VALUES(2,TO_DATE('2026-01-15', 'YYYY-MM-DD'),TO_TIMESTAMP('2026-01-15 18:30:00', 'YYYY-MM-DD HH24:MI:SS'));
1 row created.

SQL> INSERT into Show(show_id,show_date,show_time)
      2 VALUES(3,TO_DATE('2026-01-08', 'YYYY-MM-DD'),TO_TIMESTAMP('2026-01-08 13:00:00', 'YYYY-MM-DD HH24:MI:SS'));
1 row created.

SQL> INSERT into Show(show_id,show_date,show_time)
      2 VALUES(4,TO_DATE('2026-01-04', 'YYYY-MM-DD'),TO_TIMESTAMP('2026-01-04 09:30:00', 'YYYY-MM-DD HH24:MI:SS'));
1 row created.

SQL> INSERT into Show(show_id,show_date,show_time)
      2 VALUES(5,TO_DATE('2026-01-10', 'YYYY-MM-DD'),TO_TIMESTAMP('2026-01-10 10:00:00', 'YYYY-MM-DD HH24:MI:SS'));
1 row created.

```

Figure 28: Inserting of data in Show table

- Output

```

SQL> select * from Show;

  SHOW_ID SHOW_DATE SHOW_TIME
-----
      1 07-JUN-04 07-JUN-04
      2 15-JAN-26 15-JAN-26
      3 08-JAN-26 08-JAN-26
      4 04-JAN-26 04-JAN-26
      5 10-JAN-26 10-JAN-26

```

Figure 29: Output of 'Show' table

- Ticket table

```
SQL> INSERT into Ticket(ticket_id,ticket_date)
2 VALUES(1,TO_DATE('2026-01-05', 'YYYY-MM-DD'));

1 row created.

SQL> INSERT into Ticket(ticket_id,ticket_date)
2 VALUES(2,TO_DATE('2026-01-15', 'YYYY-MM-DD'));

1 row created.

SQL> INSERT into Ticket(ticket_id,ticket_date)
2 VALUES(3,TO_DATE('2026-01-08', 'YYYY-MM-DD'));

1 row created.

SQL> INSERT into Ticket(ticket_id,ticket_date)
2 VALUES(4,TO_DATE('2026-01-04', 'YYYY-MM-DD'));

1 row created.

SQL> INSERT into Ticket(ticket_id,ticket_date)
2 VALUES(5,TO_DATE('2026-01-10', 'YYYY-MM-DD'));

1 row created.
```

Figure 30: Inserting of data in Ticket table

- Output

TICKET_ID	TICKET_DA
1	05-JAN-26
2	15-JAN-26
3	08-JAN-26
4	04-JAN-26
5	10-JAN-26

Figure 31: Output of 'Ticket' table

- User_Movie Table

```

SQL> INSERT INTO User_Movie(movie_id,user_id)
2 VALUES(1,1);

1 row created.

SQL> INSERT INTO User_Movie(movie_id,user_id)
2 VALUES(1,2);

1 row created.

SQL> INSERT INTO User_Movie(movie_id,user_id)
2 VALUES(3,4);

1 row created.

SQL> INSERT INTO User_Movie(movie_id,user_id)
2 VALUES(4,5);

1 row created.

SQL> INSERT INTO User_Movie(movie_id,user_id)
2 VALUES(2,3);

1 row created.

```

Figure 32: Inserting data into 'User_Movie' table

- Output

```

SQL> select * from User_Movie;

  USER_ID  MOVIE_ID
-----
         1         1
         2         1
         3         2
         4         3
         5         4

```

Figure 33: Output of User_Movie table

- Theater_Movie_User table

```

SQL> INSERT INTO Theater_Movie_User(theater_id,movie_id,user_id)
2 VALUES(1,1,2);

1 row created.

```

Figure 34: Insertion of data in 'Theater_Movie_User' table

```

SQL> INSERT INTO Theater_Movie_User(theater_id,movie_id,user_id)
2 VALUES(2,2,3);

1 row created.

SQL> INSERT INTO Theater_Movie_User(theater_id,movie_id,user_id)
2 VALUES(3,4,5);

1 row created.

SQL> INSERT INTO Theater_Movie_User(theater_id,movie_id,user_id)
2 VALUES(5,4,2);

1 row created.

SQL> INSERT INTO Theater_Movie_User(theater_id,movie_id,user_id)
2 VALUES(4,1,1);

1 row created.

```

Figure 35: Insertion of data

- Output

```

SQL> select * from Theater_Movie_User;

THEATER_ID  MOVIE_ID  USER_ID
-----
          1          1          2
          2          2          3
          3          4          5
          5          4          2
          4          1          1

```

Figure 36: Outout of 'Theater_Movie_User'

- HTMU table

```

SQL> INSERT INTO HTMU(hall_id,theater_id,movie_id,user_id)
2 VALUES(2,5,5,4);

1 row created.

```

Figure 37: Inserting data into 'HTMU' table

```

SQL> INSERT INTO HTMU(hall_id,theater_id,movie_id,user_id)
2  VALUES(3,3,5,1);

1 row created.

SQL> INSERT INTO HTMU(hall_id,theater_id,movie_id,user_id)
2  VALUES(4,2,3,1);

1 row created.

SQL> INSERT INTO HTMU(hall_id,theater_id,movie_id,user_id)
2  VALUES(5,2,3,4);

1 row created.

SQL> INSERT INTO HTMU(hall_id,theater_id,movie_id,user_id)
2  VALUES(1,4,3,2);

1 row created.

```

Figure 38: Insert data into 'HTMU' table

- Output

```

SQL> select * from HTMU;

  HALL_ID THEATER_ID  MOVIE_ID  USER_ID
-----
        1         4         3         2
        2         5         5         4
        3         3         5         1
        4         2         3         1
        5         2         3         4

```

Figure 39: Output of 'HTMU' table

- SHTMU table

```

SQL> INSERT INTO SHTMU(show_id,hall_id,theatre_id,movie_id,user_id)
2  VALUES(5,1,1,2,2);

1 row created.

SQL> INSERT INTO SHTMU(show_id,hall_id,theatre_id,movie_id,user_id)
2  VALUES(4,3,1,3,4);

1 row created.

SQL> INSERT INTO SHTMU(show_id,hall_id,theatre_id,movie_id,user_id)
2  VALUES(3,2,1,5,4);

1 row created.

SQL> INSERT INTO SHTMU(show_id,hall_id,theatre_id,movie_id,user_id)
2  VALUES(2,2,3,4,4);

1 row created.

SQL> INSERT INTO SHTMU(show_id,hall_id,theatre_id,movie_id,user_id)
2  VALUES(1,2,5,5,4);

1 row created.

```

Figure 40: Inserting data into 'SHTMU' table

- Output

```
SQL> select * from SHTMU;
```

SHOW_ID	HALL_ID	THEATRE_ID	MOVIE_ID	USER_ID
1	2	5	5	4
2	2	3	4	4
3	2	1	5	4
4	3	1	3	4
5	1	1	2	2

Figure 41: Output of 'SHTMU' table

- TSHTMU table

```
SQL> INSERT INTO TSHTMU(ticket_id,show_id,hall_id,theater_id,movie_id,user_id)
2 VALUES(1,1,1,2,2,2)
3 ;

1 row created.

SQL> INSERT INTO TSHTMU(ticket_id,show_id,hall_id,theater_id,movie_id,user_id)
2 VALUES(2,2,3,3,4,4);

1 row created.

SQL> INSERT INTO TSHTMU(ticket_id,show_id,hall_id,theater_id,movie_id,user_id)
2 VALUES(3,3,3,4,4,4);

1 row created.

SQL> INSERT INTO TSHTMU(ticket_id,show_id,hall_id,theater_id,movie_id,user_id)
2 VALUES(4,5,1,2,3,4);

1 row created.

SQL> INSERT INTO TSHTMU(ticket_id,show_id,hall_id,theater_id,movie_id,user_id)
2 VALUES(5,5,1,1,3,3);

1 row created.
```

Figure 42: Inserting data into 'TSHTMU' table

- Output

```
SQL> select * from TSHTMU;
```

TICKET_ID	SHOW_ID	HALL_ID	MOVIE_ID	THEATER_ID	USER_ID
1	1	1	2	2	2
2	2	3	4	3	4
3	3	3	4	4	4
5	5	1	3	1	3
4	5	1	3	2	4

Figure 43: Output of 'TSHTMU' table

10 Forms

10.1 Home page

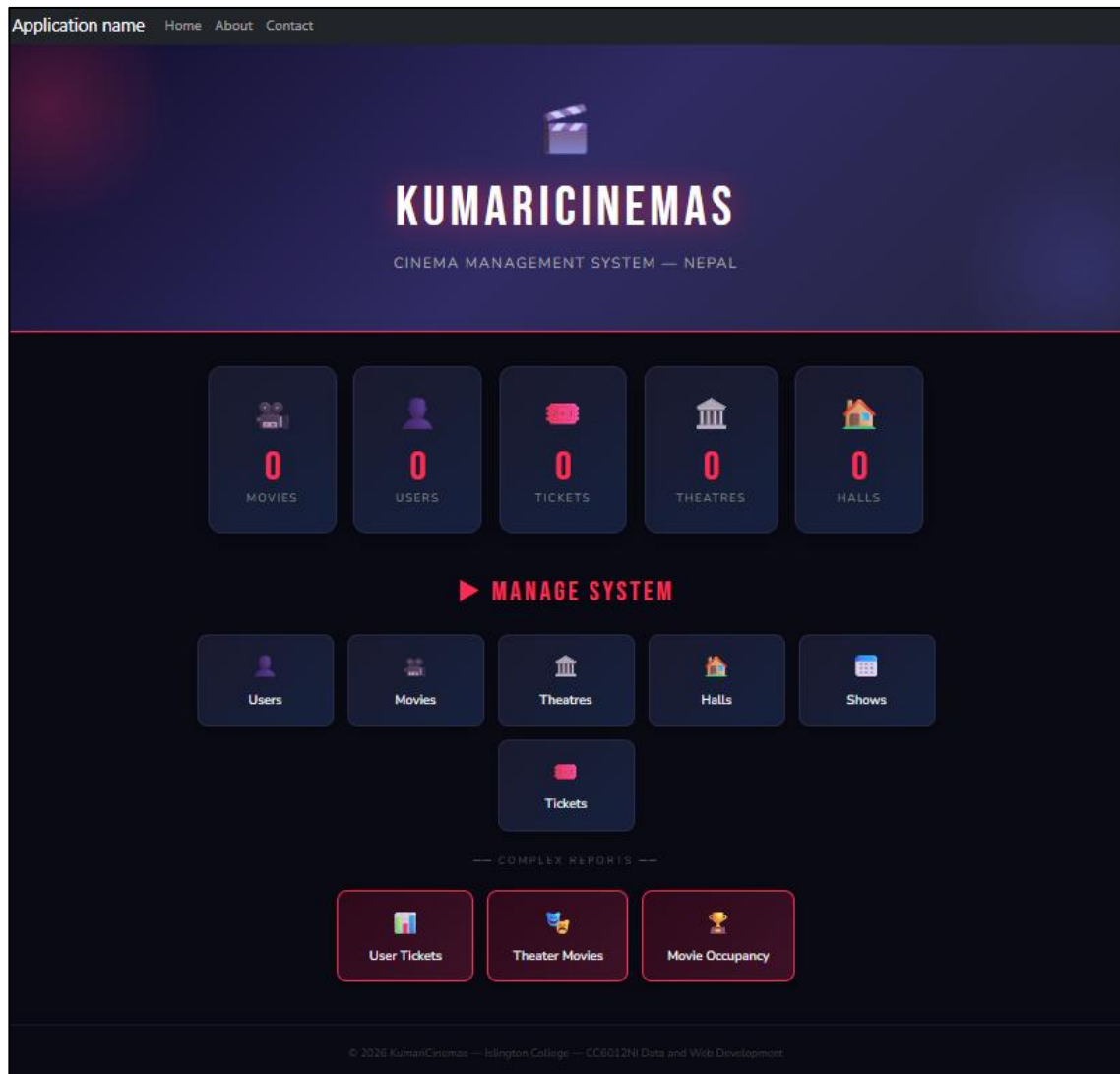


Figure 44: Home page

10.2 Complex forms and queries

10.2.1 SQL Queries

- **User Ticket**(for any user, show details of the user and the tickets he had bought during a period of six month.)

SQL statement:

```
SELECT U.USER_NAME, U.ADDRESS, T.TICKET_ID, T.TICKET_DATE,
S.SHOW_DATE, S.SHOW_TIME, M.TITLE
FROM USERS U
JOIN TSHTMU TS ON U.USER_ID = TS.USER_ID
JOIN TICKET T ON TS.TICKET_ID = T.TICKET_ID
JOIN SHOW S ON TS.SHOW_ID = S.SHOW_ID
JOIN MOVIE M ON TS.MOVIE_ID = M.MOVIE_ID
WHERE U.USER_ID = :USER_ID
```

Figure 45: Complex query 1

- **TheaterCityHall Movie** (For any theatercityhall, show the detail of movie and showtime)

SQL statement:

```
SELECT TH.THEATER_NAME, TH.THEATER_CITY, H.HALL_NAME,
H.HALL_CAPACITY, M.TITLE, M.GENRE,
S.SHOW_DATE, S.SHOW_TIME
FROM THEATRE TH
JOIN SHTMU ST ON ST.THEATRE_ID = TH.THEATER_ID
JOIN HALL H ON ST.HALL_ID = H.HALL_ID
JOIN MOVIE M ON ST.MOVIE_ID = M.MOVIE_ID
JOIN SHOW S ON ST.SHOW_ID = S.SHOW_ID
WHERE TH.THEATER_ID = :THEATER_ID
```

Figure 46: Complex Query 2

- **MovieTheatherCityHallOccupancyPerformer** (for any movie, show the details of top 3 theatercityhall who has maximum seat occupancy based on percentage).
NOTE: Only paid tickets would be counted as seat occupancy.

SQL statement:

```
SELECT TH.THEATER_NAME, TH.THEATER_CITY, H.HALL_NAME,
H.HALL_CAPACITY, COUNT(T.TICKET_ID) AS TICKETS_SOLD,
ROUND((COUNT(T.TICKET_ID)/H.HALL_CAPACITY)*100,2) AS OCCUPANCY_PCT
FROM THEATRE TH
JOIN TSHTMU TS ON TH.THEATER_ID = TS.THEATER_ID
JOIN HALL H ON TS.HALL_ID = H.HALL_ID
JOIN TICKET T ON TS.TICKET_ID = T.TICKET_ID
WHERE TS.MOVIE_ID = :MOVIE_ID
GROUP BY TH.THEATER_NAME, TH.THEATER_CITY, H.HALL_NAME, H.HALL_CAPACITY
ORDER BY OCCUPANCY_PCT DESC
```

Figure 47: Complex Query 3

10.2.2 Complex forms

USER_NAME	ADDRESS	TICKET_ID	TICKET_DATE	SHOW_DATE	SHOW_TIME	TITLE
Riwaj	Balaju	1	1/5/2026 12:00:00 AM	6/7/2004 12:00:00 AM	6/7/2004 6:30:00 PM	It Ends with us

Figure 48: Complex form 1

USER_NAME	ADDRESS	TICKET_ID	TICKET_DATE	SHOW_DATE	SHOW_TIME	TITLE
Aasra	NewRoad	5	1/10/2026 12:00:00 AM	1/10/2026 12:00:00 AM	1/10/2026 10:00:00 AM	NoteBook

Figure 49: Complex form 1

USER_NAME	ADDRESS	TICKET_ID	TICKET_DATE	SHOW_DATE	SHOW_TIME	TITLE
Simran	Thamel	2	1/15/2026 12:00:00 AM	1/15/2026 12:00:00 AM	1/15/2026 6:30:00 PM	Batman
Simran	Thamel	3	1/8/2026 12:00:00 AM	1/8/2026 12:00:00 AM	1/8/2026 1:00:00 PM	Batman
Simran	Thamel	4	1/4/2026 12:00:00 AM	1/10/2026 12:00:00 AM	1/10/2026 10:00:00 AM	NoteBook

Figure 50: Complex form 1

THEATER_NAME	THEATER_CITY	HALL_NAME	HALL_CAPACITY	TITLE	GENRE	SHOW_DATE	SHOW_TIME
Kumari Complex	KamalPokari	B	80	Harry Potter	Fantasy	1/8/2026 12:00:00 AM	1/8/2026 1:00:00 PM
Kumari Complex	KamalPokari	C	100	NoteBook	Rom-Com	1/4/2026 12:00:00 AM	1/4/2026 9:30:00 AM
Kumari Complex	KamalPokari	A	60	It Ends with us	Romantic-Drama	1/10/2026 12:00:00 AM	1/10/2026 10:00:00 AM

Figure 51: Complex form 2

THEATER_NAME	THEATER_CITY	HALL_NAME	HALL_CAPACITY	TITLE	GENRE	SHOW_DATE	SHOW_TIME
One Cinemas	Kalimati	B	80	Batman	Action	1/15/2026 12:00:00 AM	1/15/2026 6:30:00 PM

Figure 52: Complex form 2

BishwoJyoti Cineplex - Jamal							
THEATER_NAME	THEATER_CITY	HALL_NAME	HALL_CAPACITY	TITLE	GENRE	SHOW_DATE	SHOW_TIME
BishwoJyoti Cineplex	Jamal	B	80	Harry Potter	Fantasy	6/7/2004 12:00:00 AM	6/7/2004 6:30:00 PM

Figure 53: Complex form 2

It Ends with us					
THEATER_NAME	THEATER_CITY	HALL_NAME	HALL_CAPACITY	TICKETS_SOLD	OCCUPANCY_PCT
QFX	Pulchowk	A	60	1	1.67

Figure 54: Complex form 3

NoteBook					
THEATER_NAME	THEATER_CITY	HALL_NAME	HALL_CAPACITY	TICKETS_SOLD	OCCUPANCY_PCT
Kumari Complex	KamalPokari	A	60	1	1.67
QFX	Pulchowk	A	60	1	1.67

Figure 55: Complex form 3

Batman					
THEATER_NAME	THEATER_CITY	HALL_NAME	HALL_CAPACITY	TICKETS_SOLD	OCCUPANCY_PCT
One Cinemas	Kalimati	C	100	1	1
One Cinemas	Baneshwor	C	100	1	1

Figure 56: Complex form 3

10.2.3 Simple forms

10.2.3.1 User

	USER_ID	USER_NAME	ADDRESS
Edit Delete	1	Simanka	Thaiba
Edit Delete	2	Riwaj	Balaju
Edit Delete	3	Aasra	NewRoad
Edit Delete	4	Simran	Thamel
Edit Delete	5	Sam	Asan
Edit Delete	6	Tarun	Bhadgaon
Insert			

Figure 57: User details form

10.2.3.2 Theatercityhall

10.2.3.3 Show

	SHOW_ID	SHOW_DATE	SHOW_TIME
Edit Delete	1	6/7/2004 12:00:00 AM	6/7/2004 6:30:00 PM
Edit Delete	2	1/15/2026 12:00:00 AM	1/15/2026 6:30:00 PM
Edit Delete	3	1/8/2026 12:00:00 AM	1/8/2026 1:00:00 PM
Edit Delete	4	1/4/2026 12:00:00 AM	1/4/2026 9:30:00 AM
Edit Delete	5	1/10/2026 12:00:00 AM	1/10/2026 10:00:00 AM
Insert			

Figure 58: Show details form

10.2.3.4 Movie

	MOVIE_ID	TITLE	LANGUAGE	DURATION	GENRE	RELEASE_DATE
Edit Delete	1	Mean girls	English	1.50	Rom-Com	1/5/2002 12:00:00 AM
Edit Delete	2	It Ends with us	English	2.50	Romantic-Drama	1/15/2026 12:00:00 AM
Edit Delete	3	NoteBook	English	2	Rom-Com	1/8/2026 12:00:00 AM
Edit Delete	4	Batman	English	2.50	Action	1/4/2026 12:00:00 AM
Edit Delete	5	Harry Potter	English	2	Fantasy	1/10/2026 12:00:00 AM
Insert						

Figure 59: Movie details form

10.2.3.5 Ticket

	TICKET_ID	TICKET_DATE
Edit Delete	1	1/5/2026 12:00:00 AM
Edit Delete	2	1/15/2026 12:00:00 AM
Edit Delete	3	1/8/2026 12:00:00 AM
Edit Delete	4	1/4/2026 12:00:00 AM
Edit Delete	5	1/10/2026 12:00:00 AM
Insert		

Figure 60: Ticket details form

11 User manual

- Section 1: Users Form

	USER_ID	USER_NAME	ADDRESS
Edit Delete	1	Simanka	Thaiba
Edit Delete	2	Riwaj	Balaju
Edit Delete	3	Aasra	NewRoad
Edit Delete	4	Simran	Thamel
Edit Delete	5	Sam	Asan
Delete	6	Tarun	Bhadgaon
Insert			

Clicking edit to modify

Clicking delete for deletion

Clicking insert for new entry

Figure 61: User list

- Data insertion:

USER_ID:

USER_NAME:

ADDRESS:

[Insert](#) [Cancel](#)

Figure 62: User form data insertion

- Section 2: Movie Form

	MOVIE_ID	TITLE	LANGUAGE	DURATION	GENRE	RELEASE_DATE
Edit Delete	1	Mean girls	English	1.50	Rom-Com	1/5/2002 12:00:00 AM
Edit Delete	2	It Ends with us	English	2.50	Romantic-Drama	1/15/2026 12:00:00 AM
Edit Delete	3	NoteBook	English	2	Rom-Com	1/8/2026 12:00:00 AM
Edit Delete	4	Batman	English	2.50	Action	1/4/2026 12:00:00 AM
Delete	5	Harry Potter	English	2	Fantasy	1/10/2026 12:00:00 AM
Insert						

Clicking edit to modify

Clicking insert for new entry

Figure 63: Movie form

MOVIE_ID:

TITLE:

LANGUAGE:

DURATION:

GENRE:

RELEASE_DATE:

[Insert](#) [Cancel](#)

Figure 64: Inserting into movie form

- Section 3: Theatre Form

	THEATER_ID	THEATER_NAME	THEATER_CITY
Edit Delete	1	Kumari Complex	KamalPokari
Edit Delete	2	QFX	Pulchowk
Edit Delete	3	One Cinemas	Kalimati
Edit Delete	4	One Cinemas	Baneshwor
Delete	5	Bishwojyoti Cineplex	Jamal
Insert			

Clicking edit to modify

Clicking delete for deletion

Clicking insert for new entry

Figure 65: Theater form list

THEATER_ID:

THEATER_NAME:

THEATER_CITY:

[Insert](#) [Cancel](#)

Figure 66: Insertion of new theater data

- Section 4: Hall Form

	HALL_ID	HALL_CAPACITY	HALL_NAME
Edit Delete	1	60	A
Edit Delete	2	80	B
Edit Delete	3	100	C
Edit Delete	4	110	D
Edit Delete	5	120	E
Insert			

- Section 5: Show Form

	SHOW_ID	SHOW_DATE	SHOW_TIME
Edit Delete	1	6/7/2004 12:00:00 AM	6/7/2004 6:30:00 PM
Edit Delete	2	1/15/2026 12:00:00 AM	1/15/2026 6:30:00 PM
Edit Delete	3	1/8/2026 12:00:00 AM	1/8/2026 1:00:00 PM
Edit Delete	4	1/4/2026 12:00:00 AM	1/4/2026 9:30:00 AM
Edit Delete	5	1/10/2026 12:00:00 AM	1/10/2026 10:00:00 AM
Insert			

- Section 6: Ticket Form

	TICKET_ID	TICKET_DATE
Edit Delete	1	1/5/2026 12:00:00 AM
Edit Delete	2	1/15/2026 12:00:00 AM
Edit Delete	3	1/8/2026 12:00:00 AM
Edit Delete	4	1/4/2026 12:00:00 AM
Edit Delete	5	1/10/2026 12:00:00 AM
Insert		

12 Testing

12.1 User form

12.1.1 Insert

Before:

	USER_ID	USER_NAME	ADDRESS
Edit Delete	1	Simanka	Thaiba
Edit Delete	2	Riwaj	Balaju
Edit Delete	3	Aasra	NewRoad
Edit Delete	4	Simran	Thamel
Edit Delete	5	Sam	Asan
Edit Delete	6	Tarun	Bhadgaon
Insert			

Figure 67: User form before insertion

During:

USER_ID:
 USER_NAME:
 ADDRESS:

[Insert](#) [Cancel](#)

Figure 68: Inserting data in user form

After:

	USER_ID	USER_NAME	ADDRESS
Edit Delete	1	Simanka	Thaiba
Edit Delete	2	Riwaj	Balaju
Edit Delete	3	Aasra	NewRoad
Edit Delete	4	Simran	Thamel
Edit Delete	5	Sam	Asan
Edit Delete	6	Tarun	Bhadgaon
Edit Delete	7	Kamala	Thaiba
insert			

Figure 69: User form after data insertion

12.1.2 Update

Before:

	USER_ID	USER_NAME	ADDRESS
Edit Delete	1	Simanka	Thaiba
Edit Delete	2	Riwaj	Balaju
Edit Delete	3	Aasra	NewRoad
Edit Delete	4	Simran	Thamel
Edit Delete	5	Sam	Asan
Edit Delete	6	Tarun	Bhadgaon
Edit Delete	7	Kamala	Thaiba
insert			

Figure 70: User form before update

During:

Update Cancel	6	Trisha	Bhadgaon
---	---	-------------------------------------	---------------------------------------

Figure 71: User form during update

After:

	USER_ID	USER_NAME	ADDRESS
Edit Delete	1	Simanka	Thaiba
Edit Delete	2	Riwaj	Balaju
Edit Delete	3	Aasra	NewRoad
Edit Delete	4	Simran	Thamel
Edit Delete	5	Sam	Asan
Edit Delete	6	Trisha	Bhadgaon
Edit Delete	7	Kamala	Thaiba
insert			

Figure 72: User form after update

12.1.3 Delete

Before:

	USER_ID	USER_NAME	ADDRESS
Edit Delete	1	Simanka	Thaiba
Edit Delete	2	Riwaj	Balaju
Edit Delete	3	Aasra	NewRoad
Edit Delete	4	Simran	Thamel
Edit Delete	5	Sam	Asan
Edit Delete	6	Trisha	Bhadgaon
Edit Delete	7	Kamala	Thaiba
Insert			

Figure 73: User form before deletion

After:

	USER_ID	USER_NAME	ADDRESS
Edit Delete	1	Simanka	Thaiba
Edit Delete	2	Riwaj	Balaju
Edit Delete	3	Aasra	NewRoad
Edit Delete	4	Simran	Thamel
Edit Delete	5	Sam	Asan
Edit Delete	6	Trisha	Bhadgaon
Insert			

Figure 74: User form after deletion

12.2 Movie form

12.2.1 Insert

Before:

	MOVIE_ID	TITLE	LANGUAGE	DURATION	GENRE	RELEASE_DATE
Edit Delete	1	Mean girls	English	1.50	Rom-Com	1/5/2002 12:00:00 AM
Edit Delete	2	It Ends with us	English	2.50	Romantic-Drama	1/15/2026 12:00:00 AM
Edit Delete	3	NoteBook	English	2	Rom-Com	1/8/2026 12:00:00 AM
Edit Delete	4	Batman	English	2.50	Action	1/4/2026 12:00:00 AM
Edit Delete	5	Harry Potter	English	2	Fantasy	1/10/2026 12:00:00 AM
Insert						

Figure 75: Movie form before insertion

During:

MOVIE_ID:
 TITLE:
 LANGUAGE:
 DURATION:
 GENRE:
 RELEASE_DATE:
[Insert](#) [Cancel](#)

Figure 76: Movie form during insertion

After:

	MOVIE_ID	TITLE	LANGUAGE	DURATION	GENRE	RELEASE_DATE
Edit Delete	1	Mean girls	English	1.50	Rom-Com	1/5/2002 12:00:00 AM
Edit Delete	2	It Ends with us	English	2.50	Romantic-Drama	1/15/2026 12:00:00 AM
Edit Delete	3	NoteBook	English	2	Rom-Com	1/8/2026 12:00:00 AM
Edit Delete	4	Batman	English	2.50	Action	1/4/2026 12:00:00 AM
Edit Delete	5	Harry Potter	English	2	Fantasy	1/10/2026 12:00:00 AM
Edit Delete	6	Spiderman	English	1.50	Sci-fi	1/4/2026 12:00:00 AM
Insert						

Figure 77: Movie form after insertion

12.2.2 Update

Before:

	MOVIE_ID	TITLE	LANGUAGE	DURATION	GENRE	RELEASE_DATE
Edit Delete	1	Mean girls	English	1.50	Rom-Com	1/5/2002 12:00:00 AM
Edit Delete	2	It Ends with us	English	2.50	Romantic-Drama	1/15/2026 12:00:00 AM
Edit Delete	3	NoteBook	English	2	Rom-Com	1/8/2026 12:00:00 AM
Edit Delete	4	Batman	English	2.50	Action	1/4/2026 12:00:00 AM
Edit Delete	5	Harry Potter	English	2	Fantasy	1/10/2026 12:00:00 AM
Edit Delete	6	Spiderman	English	1.50	Sci-fi	1/4/2026 12:00:00 AM
Insert						

Figure 78: Movie form before update

During:

[Update](#) [Cancel](#) 4

Figure 79: Movie form during update

After:

	MOVIE_ID	TITLE	LANGUAGE	DURATION	GENRE	RELEASE_DATE
Edit Delete	1	Mean girls	English	1.50	Rom-Com	1/5/2002 12:00:00 AM
Edit Delete	2	It Ends with us	English	2.50	Romantic-Drama	1/15/2026 12:00:00 AM
Edit Delete	3	NoteBook	English	2	Rom-Com	1/8/2026 12:00:00 AM
Edit Delete	4	Batman	English	2	Action	1/4/2026 12:00:00 AM
Edit Delete	5	Harry Potter	English	2	Fantasy	1/10/2026 12:00:00 AM
Edit Delete	6	Spiderman	English	1.50	Sci-fi	1/4/2026 12:00:00 AM
insert						

Figure 80: Movie form after update

12.2.3 Delete

Before:

	MOVIE_ID	TITLE	LANGUAGE	DURATION	GENRE	RELEASE_DATE
Edit Delete	1	Mean girls	English	1.50	Rom-Com	1/5/2002 12:00:00 AM
Edit Delete	2	It Ends with us	English	2.50	Romantic-Drama	1/15/2026 12:00:00 AM
Edit Delete	3	NoteBook	English	2	Rom-Com	1/8/2026 12:00:00 AM
Edit Delete	4	Batman	English	2	Action	1/4/2026 12:00:00 AM
Edit Delete	5	Harry Potter	English	2	Fantasy	1/10/2026 12:00:00 AM
Edit Delete	6	Spiderman	English	1.50	Sci-fi	1/4/2026 12:00:00 AM
insert						

Figure 81: Movie form before deletion

After:

12.3 Theater form

12.3.1 Insert

Before:

	THEATER_ID	THEATER_NAME	THEATER_CITY
Edit Delete	1	Kumari Complex	KamalPokari
Edit Delete	2	QFX	Pulchowk
Edit Delete	3	One Cinemas	Kalimati
Edit Delete	4	One Cinemas	Baneshwor
Edit Delete	5	BishwoJyoti Cineplex	Jamal
insert			

Figure 82: Theater form before insertion

During:

THEATER_ID:

THEATER_NAME:

THEATER_CITY:

[Insert](#) [Cancel](#)

Figure 83: Theater form during insertion

After:

	THEATER_ID	THEATER_NAME	THEATER_CITY
Edit Delete	1	Kumari Complex	KamalPokari
Edit Delete	2	QFX	Pulchowk
Edit Delete	3	One Cinemas	Kalimati
Edit Delete	4	One Cinemas	Baneshwor
Edit Delete	5	Bishwojyoti Cineplex	Jamal
Edit Delete	6	Mata Complex	Baneshwor
Insert			

Figure 84: Theater form after insertion

12.3.2 Update

Before:

	THEATER_ID	THEATER_NAME	THEATER_CITY
Edit Delete	1	Kumari Complex	KamalPokari
Edit Delete	2	QFX	Pulchowk
Edit Delete	3	One Cinemas	Kalimati
Edit Delete	4	One Cinemas	Baneshwor
Edit Delete	5	Bishwojyoti Cineplex	Jamal
Edit Delete	6	Mata Complex	Baneshwor
Insert			

Figure 85: Theater form before update

During:

[Update](#) [Cancel](#)

4

Figure 86: Theater form during update

After:

	THEATER_ID	THEATER_NAME	THEATER_CITY
Edit Delete	1	Kumari Complex	KamalPokari
Edit Delete	2	QFX	Pulchowk
Edit Delete	3	One Cinemas	Kalimati
Edit Delete	4	Thrice Cinemas	Baneshwor
Edit Delete	5	Bishwojyoti Cineplex	Jamal
Edit Delete	6	Mata Complex	Baneshwor
Insert			

Figure 87: Theater form after update

12.3.3 Delete

Before:

	THEATER_ID	THEATER_NAME	THEATER_CITY
Edit Delete	1	Kumari Complex	KamalPokari
Edit Delete	2	QFX	Pulchowk
Edit Delete	3	One Cinemas	Kalimati
Edit Delete	4	Thrice Cinemas	Baneshwor
Edit Delete	5	Bishwojyoti Cineplex	Jamal
Edit Delete	6	Mata Complex	Baneshwor
Insert			

Figure 88: Theater form before deletion

After:

	THEATER_ID	THEATER_NAME	THEATER_CITY
Edit Delete	1	Kumari Complex	KamalPokari
Edit Delete	2	QFX	Pulchowk
Edit Delete	3	One Cinemas	Kalimati
Edit Delete	4	Thrice Cinemas	Baneshwor
Edit Delete	5	Bishwojyoti Cineplex	Jamal
Insert			

Figure 89: Theater form after deletion

12.4 Hall form

12.4.1 Insert

Before:

	HALL_ID	HALL_CAPACITY	HALL_NAME
Edit Delete	1	60	A
Edit Delete	2	80	B
Edit Delete	3	100	C
Edit Delete	4	110	D
Edit Delete	5	120	E
Insert			

Figure 90: Hall form before insertion

During:

HALL_ID:	6
HALL_CAPACITY:	67
HALL_NAME:	F
Insert Cancel	

Figure 91: Hall form during insertion

After:

	HALL_ID	HALL_CAPACITY	HALL_NAME
Edit Delete	1	60	A
Edit Delete	2	80	B
Edit Delete	3	100	C
Edit Delete	4	110	D
Edit Delete	5	120	E
Edit Delete	6	67	F
Insert			

Figure 92: Hall form after new insertion

12.4.2 Update

Before:

	HALL_ID	HALL_CAPACITY	HALL_NAME
Edit Delete	1	60	A
Edit Delete	2	80	B
Edit Delete	3	100	C
Edit Delete	4	110	D
Edit Delete	5	120	E
Edit Delete	6	67	F
Insert			

Figure 93: Hall form before update

During:

Update Cancel	4	111	D
---	---	----------------------------------	--------------------------------

Figure 94: Hall form during update

After:

	HALL_ID	HALL_CAPACITY	HALL_NAME
Edit Delete	1	60	A
Edit Delete	2	80	B
Edit Delete	3	100	C
Edit Delete	4	111	D
Edit Delete	5	120	E
Edit Delete	6	67	F
insert			

Figure 95: Hall form after insertion

12.4.3 Delete

Before:

	HALL_ID	HALL_CAPACITY	HALL_NAME
Edit Delete	1	60	A
Edit Delete	2	80	B
Edit Delete	3	100	C
Edit Delete	4	111	D
Edit Delete	5	120	E
Edit Delete	6	67	F
insert			

Figure 96: Hall form before deletion

After:

12.5 Complex forms

12.5.1 Complex form 1

2 ▾						
USER_NAME	ADDRESS	TICKET_ID	TICKET_DATE	SHOW_DATE	SHOW_TIME	TITLE
Riwaj	Balaju	1	1/5/2026 12:00:00 AM	6/7/2004 12:00:00 AM	6/7/2004 6:30:00 PM	It Ends with us

Figure 97: Complex form 1

3 ▾						
USER_NAME	ADDRESS	TICKET_ID	TICKET_DATE	SHOW_DATE	SHOW_TIME	TITLE
Aasra	NewRoad	5	1/10/2026 12:00:00 AM	1/10/2026 12:00:00 AM	1/10/2026 10:00:00 AM	NoteBook

Figure 98: Complex form 1

4 ▾						
USER_NAME	ADDRESS	TICKET_ID	TICKET_DATE	SHOW_DATE	SHOW_TIME	TITLE
Simran	Thamel	2	1/15/2026 12:00:00 AM	1/15/2026 12:00:00 AM	1/15/2026 6:30:00 PM	Batman
Simran	Thamel	3	1/8/2026 12:00:00 AM	1/8/2026 12:00:00 AM	1/8/2026 1:00:00 PM	Batman
Simran	Thamel	4	1/4/2026 12:00:00 AM	1/10/2026 12:00:00 AM	1/10/2026 10:00:00 AM	NoteBook

Figure 99: Complex form 1

12.5.2 Complex form 2

Kumari Complex - KamalPokari ▾							
THEATER_NAME	THEATER_CITY	HALL_NAME	HALL_CAPACITY	TITLE	GENRE	SHOW_DATE	SHOW_TIME
Kumari Complex	KamalPokari	B	80	Harry Potter	Fantasy	1/8/2026 12:00:00 AM	1/8/2026 1:00:00 PM
Kumari Complex	KamalPokari	C	100	NoteBook	Rom-Com	1/4/2026 12:00:00 AM	1/4/2026 9:30:00 AM
Kumari Complex	KamalPokari	A	60	It Ends with us	Romantic-Drama	1/10/2026 12:00:00 AM	1/10/2026 10:00:00 AM

Figure 100: Complex form 2

One Cinemas - Kalimati ▾							
THEATER_NAME	THEATER_CITY	HALL_NAME	HALL_CAPACITY	TITLE	GENRE	SHOW_DATE	SHOW_TIME
One Cinemas	Kalimati	B	80	Batman	Action	1/15/2026 12:00:00 AM	1/15/2026 6:30:00 PM

Figure 101: Complex form 2

BishwoJyoti Cineplex - Jamal ▾							
THEATER_NAME	THEATER_CITY	HALL_NAME	HALL_CAPACITY	TITLE	GENRE	SHOW_DATE	SHOW_TIME
BishwoJyoti Cineplex	Jamal	B	80	Harry Potter	Fantasy	6/7/2004 12:00:00 AM	6/7/2004 6:30:00 PM

Figure 102: Complex form 2

12.5.3 Complex form 3

Figure 103: Complex form 2

It Ends with us ▾						
THEATER_NAME	THEATER_CITY	HALL_NAME	HALL_CAPACITY	TICKETS_SOLD	OCCUPANCY_PCT	
QFX	Pulchowk	A	60	1	1.67	

Figure 104: Complex form 3

NoteBook ▾						
THEATER_NAME	THEATER_CITY	HALL_NAME	HALL_CAPACITY	TICKETS_SOLD	OCCUPANCY_PCT	
Kumari Complex	KamalPokari	A	60	1	1.67	
QFX	Pulchowk	A	60	1	1.67	

Figure 105: Complex form 3

Batman ▾						
THEATER_NAME	THEATER_CITY	HALL_NAME	HALL_CAPACITY	TICKETS_SOLD	OCCUPANCY_PCT	
One Cinemas	Kalimati	C	100	1	1	
One Cinemas	Baneshwor	C	100	1	1	

Figure 106: Complex form 3

13 Further discussion

Working on this coursework provided valuable experience in understanding the process of designing and developing a database-driven system. One of the most important techniques used in this project was **database normalization**, which helped organize the data into logical tables while reducing redundancy and ensuring consistency within the database. By applying normalization rules, it was possible to clearly identify primary keys, foreign keys, and relationships between entities.

Another important tool used during the development process was **Oracle SQL Developer Data Modeler**, which was used to create both the initial and final ER diagrams based on the case study and the normalized tables. This tool also helped generate the scripts required to create the database tables. The actual implementation of the database was carried out using **Oracle SQL Developer**, where SQL commands such as CREATE, INSERT, and SELECT were used to build and test the database structure.

In addition, the development of the web application using **ASP.NET with C#** allowed practical experience in creating interactive web forms that support operations such as adding, updating, viewing, and deleting records. Testing was also an important part of the project, as different test cases were used to ensure that the forms and queries worked correctly and handled errors properly. Overall, this coursework helped develop practical skills in database design, SQL programming, and web application development, which are essential for building real-world information systems.